

Algorithmic Efficiency & Recursion Toolkit (AERT)

Abstract Data Type (ADT)

An Abstract Data Type (ADT) defines what operations can be performed without specifying how they are implemented.

Stack operations such as push, pop, and peek take constant time $O(1)$ because they operate only on the top element.

In this project, Stack ADT is used to store Tower of Hanoi moves.

Factorial Analysis

- 1 Time Complexity: $O(n)$
- 2 Space Complexity: $O(n)$
- 3 Uses recursion with base case $0! = 1$

Fibonacci Analysis

Naive Fibonacci is slow because it recalculates the same values repeatedly.

- 1 Naive Time Complexity: $O(2^n)$
- 2 Memoized Time Complexity: $O(n)$
- 3 Memoization stores previous results to improve efficiency

Tower of Hanoi

- 1 Number of moves = $2^n - 1$
- 2 Time Complexity: $O(2^n)$
- 3 Space Complexity: $O(n)$

Binary Search Analysis

Recurrence relation: $T(n) = T(n/2) + O(1)$

- 1 Time Complexity: $O(\log n)$
- 2 Best Case: $O(1)$
- 3 Worst Case: $O(\log n)$

Algorithm Paradigms

- 1 Divide & Conquer: Binary Search splits problem into halves
- 2 Dynamic Programming: Memoized Fibonacci stores computed values
- 3 Recursion: Tower of Hanoi solves smaller subproblems

Real-Life Examples

- 1 Divide & Conquer: Searching in dictionary
- 2 Dynamic Programming: Navigation apps storing routes
- 3 Greedy: Giving change using largest coins first